What You'll Walk Away With

By the end of this project, you'll have:

1. A pipeline that checks IDs at the door — your broker will require every device to prove who it is before connecting
2. A unique certificate for each sensor — each of the three HYDROLOGIC devices gets its own digital ID badge
3. Proof that it works — benchmark data showing mTLS doesn't slow things down, and test results showing unauthorized devices get rejected
4. A device provisioning policy — a clear document explaining how new devices get approved, how old ones get retired, and what happens if one is compromised

Focusing Week 5

```
Certificate Authority (CA)
├── ca.pem (public)
└── ca-key.pem (private - keep secret!)
    │
    ├── Server Certificate (from Week 4)
    │   ├── server.pem (public)
    │   └── server-key.pem (private)
    │
    └── Client Certificates (NEW in Week 5!)
        ├── device-001.pem + device-001-key.pem  ← Main Building
        ├── device-002.pem + device-002-key.pem  ← Pool/Spa Wing
        └── device-003.pem + device-003-key.pem  ← Kitchen/Laundry
```

```
certs/
├── ca.pem            # Certificate Authority (from Project 4 or auto-generated)
├── ca-key.pem        # CA private key (KEEP SECRET!)
├── server.pem        # Broker certificate (from Project 4 or auto-generated)
├── server-key.pem    # Broker private key (KEEP SECRET!)
├── device-001.pem    # MainBuilding device certificate (NEW)
├── device-001-key.pem  # MainBuilding device private key (NEW)
├── device-002.pem    # PoolSpa device certificate (NEW)
├── device-002-key.pem  # PoolSpa device private key (NEW)
├── device-003.pem    # Kitchen device certificate (NEW)
└── device-003-key.pem  # Kitchen device private key (NEW)
```

- After creating a new mTLS conf file and only changing a few things such as setting `require_certificate true` which turns on mTLS, as well as `use_identity_as_username true` which allows the broker to identify the devices connected. Below is shown an error while trying to connect with no certificate meaning it's working.

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>mosquitto_sub -h localhost -p 8883 --cafile certs/ca
.pem -t "test"
Error: The connection was lost
```

```
1776997257: New connection from ::1:61014 on port 8883.
1776997257: OpenSSL Error while trying to get the error[0]: error:0A0000C7:SSL routines::peer did not return a certifica
te
```

- Now if we were to try and connect our publisher and subscriber we would get connection errors the same as a random attacker would, the broker demands credentials so lets give it some.

```
                    try:
                    self.client.tls_set(
                    ca_certs="certs/ca.pem",
                certfile=f"certs/{self.device_id}.pem",
               keyfile=f"certs/{self.device_id}-key.pem",
                    cert_reqs=ssl.CERT_REQUIRED,
                    tls_version=ssl.PROTOCOL_TLS,
                                )
        print("[TLS] mTLS configured with client certificate")
                    except FileNotFoundError as e:
            raise RuntimeError(f"Certificate not found for
              {self.device_id}: {e}") from e
```

This is what was configured in both the subscriber and publisher script, what you see is certfile and keyfile being configured, this tells authorized certs "Hey heres my key, whats yours?" if both verify and pass the mTLS is successful and a connection is made

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python subscriber_mtls.py
============================================================
  Grand Marina Security Dashboard (mTLS)
============================================================
  Subscribing to: hydroficient/grandmarina/#
  Certificate: certs/device-001.pem
============================================================
[TLS] mTLS configured with client certificate
[CONNECTING] localhost:8883...
Connecting to broker...

============================================================
  GRAND MARINA WATER MONITORING DASHBOARD
  Connected at: 2026-04-23 20:19:02
============================================================
```

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python publisher_mtls.py
All sensors running. Press Ctrl+C to stop.
[TLS] mTLS configured with client certificate
[TLS] mTLS configured with client certificate
[TLS] mTLS configured with client certificate
Starting device: device-003
Location: kitchen
Publishing to: hydroficient/grandmarina/sensors/kitchen/readings
Interval: 2 seconds
Starting device: device-001
----------------------------------------
```

(The THREE mTLS configured lines are devices 001, 002, 003. This will be changed later to display the right devices)

- This screenshot shows what happens when certfile and keyfile are removed and a rogue device tries to connect, DENIED.

```
============================================================
C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05\publisher_mtls_test.py:110: DeprecationWarning: Callback AP
I version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id=CLIENT_NAME, **MQTT_CLIENT_ARGS)
[TLS] mTLS configured with client certificate

[CONNECTING] localhost:8883...
[ERROR] Connection refused. Is the broker running?
```

- Now we will be doing some testing again, How much is at cost due to switching to mTLS from TLS, The focus here is connection time and message latency

```
1. Client connects
2. Server sends certificate
3. Client verifies server certificate
4. Server requests client certificate      ← extra
5. Client sends certificate                ← extra
6. Server verifies client certificate      ← extra
7. Key exchange → encrypted channel open
```

Connection Latency Test

(mTLS & TLS)

```
Testing One-Way TLS (port 8883)...
C:\Users\nsain\Documents\hydroficie
sion 1 is deprecated, update to lat
  client = mqtt.Client(
  Trial 1:  5.5 ms
  Trial 2:  30.1 ms
  Trial 3:  22.1 ms
  Trial 4:  6.0 ms
  Trial 5:  22.3 ms
  Trial 6:  19.7 ms
  Trial 7:  29.4 ms
  Trial 8:  4.6 ms
  Trial 9:  4.8 ms
  Trial 10:  5.8 ms
  Trial 11:  4.9 ms
  Trial 12:  5.0 ms
  Trial 13:  6.2 ms
  Trial 14:  28.1 ms
  Trial 15:  22.7 ms
  Trial 16:  4.3 ms
  Trial 17:  5.9 ms
  Trial 18:  46.0 ms
  Trial 19:  4.5 ms
  Trial 20:  21.9 ms
```

```
Testing Mutual TLS (port 8884)...
C:\Users\nsain\Documents\hydroficient-project\
ion 1 is deprecated, update to latest version
  client = mqtt.Client(
  Trial 1: 8.2 ms
  Trial 2: 28.2 ms
  Trial 3: 3.5 ms
  Trial 4: 16.1 ms
  Trial 5: 3.9 ms
  Trial 6: 3.7 ms
  Trial 7: 4.3 ms
  Trial 8: 28.7 ms
  Trial 9: 16.0 ms
  Trial 10: 3.8 ms
  Trial 11: 3.5 ms
  Trial 12: 4.7 ms
  Trial 13: 29.1 ms
  Trial 14: 4.2 ms
  Trial 15: 27.6 ms
  Trial 16: 17.4 ms
  Trial 17: 11.8 ms
  Trial 18: 4.3 ms
  Trial 19: 27.3 ms
  Trial 20: 5.4 ms
```

(Results)

```
============================================================
  Connection Time Comparison
============================================================
  Trials: 20

  One-Way TLS:
    Average: 12.6 ms
    Min: 3.5 ms | Max: 29.1 ms

  Mutual TLS:
    Average: 15.0 ms
    Min: 4.3 ms | Max: 46.0 ms

  Overhead: +2.4 ms (+19.1%)
============================================================
```

Message Latency Test

(Results)

```
============================================================
  Message Latency Comparison
============================================================
  Messages: 50

  One-Way TLS:
    Average: 0.4 ms
    Min: 0.2 ms | Max: 1.0 ms

  Mutual TLS:
    Average: 0.5 ms
    Min: 0.2 ms | Max: 1.6 ms

  Overhead: +0.1 ms (+14.2%)
============================================================
```

- This isn't the only test I was going to conduct, next I will be fully trying to break my system, I will be trying to act as a hacker and try to connect to the system without a certificate, an expired one, a fake one. This is the only way to see if my pipeline is really secure.

Test 1: *Verifying normal mTLS connections work.*

```
python identity_tester.py --mode test-correct
```

- This file runs tests to check if only the correct certificates are verified

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python identity_tester.py --mode test-correct
============================================================
IDENTITY ATTACK SIMULATION
============================================================

------------------------------------------------------------
SCENARIO A: Correct Client Certificate
------------------------------------------------------------
Testing connection with valid device-001 certificate...
C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05\identity_tester.py:266: DeprecationWarning: Callback API ve
rsion 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="test-correct-cert", **MQTT_CLIENT_ARGS)

============================================================
TEST: Valid Client Certificate
============================================================
Expected: Connection succeeds
Actual:   CONNECTION SUCCEEDED

>>> TEST PASSED - Connection succeeded as expected <<<
```

This test was a success, The main goal here is to test if a valid certificate is able to connect their broker.

—----------

Test 2: *Verify brokers are rejected without certificates*

```
python identity_tester.py --mode test-no-cert
```

- File is the same, command we are running this time is going to test brokers with no certificate

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python identity_tester.py --mode test-no-cert
============================================================
IDENTITY ATTACK SIMULATION
============================================================

------------------------------------------------------------
SCENARIO B: No Client Certificate
------------------------------------------------------------
Testing connection WITHOUT any client certificate...
C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05\identity_tester.py:305: DeprecationWarning: Callback API ve
rsion 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="test-no-cert", **MQTT_CLIENT_ARGS)

============================================================
TEST: No Client Certificate
============================================================
Expected: Connection rejected
Actual:   CONNECTION FAILED
Error:    Connection failed with rc=-1

>>> TEST PASSED - Connection was correctly rejected <<<
```

As we see here the connection was rejected due to a broker having no certificate.

—----------

Test 3: *Certificate from wrong CA*

- *This test will make sure only certificates signed by our Root CA are accepted.*

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python identity_tester.py --mode test-wrong-ca
============================================================
IDENTITY ATTACK SIMULATION
============================================================

------------------------------------------------------------
SCENARIO C: Certificate from Wrong CA
------------------------------------------------------------
Testing connection with certificate signed by DIFFERENT CA...
C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05\identity_tester.py:350: DeprecationWarning: Callback API ve
rsion 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="test-wrong-ca", **MQTT_CLIENT_ARGS)

============================================================
TEST: Wrong CA Certificate
============================================================
Expected: Connection rejected
Actual:   CONNECTION FAILED
Error:    Connection failed with rc=-1

>>> TEST PASSED - Connection was correctly rejected <<<
```

Same as Test 2 but this time it does have a valid certificate but from a different CA outside of our system so therefore its rejected.

Test 4: *Expired certificates*

- *This tests if certificates are expired.*

```
(base) C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05>python identity_tester.py --mode test-expired
============================================================
IDENTITY ATTACK SIMULATION
============================================================

------------------------------------------------------------
SCENARIO D: Expired Certificate
------------------------------------------------------------
Testing connection with EXPIRED client certificate...
C:\Users\nsain\Documents\hydroficient-project\Week-04 and 05\identity_tester.py:403: DeprecationWarning: Callback API ve
rsion 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="test-expired", **MQTT_CLIENT_ARGS)

============================================================
TEST: Expired Certificate
============================================================
Expected: Connection rejected
Actual:   CONNECTION FAILED
Error:    Connection failed with rc=-1

>>> TEST PASSED - Connection was correctly rejected <<<
```

Very simple test that tries to detect if any certificate is expired potentially requesting a renewal from the CA